From Bug Fixes to Feature Pull Requests: An Industrial Audit of Coding Agents and a Case for Harness Engineering

Zihan Wu*
Department of Electrical Engineering, City University of Hong Kong
Hong Kong, China

Yun Peng*,†
Department of Computer Science and Engineering, The Chinese University of Hong Kong
Hong Kong, China
ypeng@cse.cuhk.edu.hk

Chun Yong Chong
School of Information Technology, Monash University Malaysia
Subang Jaya, Malaysia

Xin Zhou
Singapore Management University
Singapore, Singapore

Zeyang Zhuang
The Chinese University of Hong Kong
Hong Kong, China

Rui Shu
Independent Researcher
Singapore, Singapore

Kui Liu
Huawei Technologies Co., Ltd.
Shenzhen, China

Abstract

Frontier coding agents now solve over 70% of SWE-bench bug-fix tasks, prompting industry teams to ask whether they are ready to deliver *feature pull requests*—multi-file, API-bearing, test-and-doc-bundled changes that engineers actually merge. We report an industrial-scale assessment that addresses this deployment question for Huawei's coding-agent rollout, evaluating four commercial agents (Claude Opus 4.6, GPT-5.4 / Codex, Composer-2 / Cursor, GLM-5.1 / OpenCode) on 62 feature-implementation tasks: 12 internal Huawei tasks (CANN operator kernels, MindSpeed training, torch_npu, and a Kubernetes workflow) and 50 open-source tasks from CapBench (Kafka, CPython, Airflow, TiDB, Elasticsearch, and others). Each agent is run with both short (~3-line) and long (~30-line) requirement prompts, producing $4 \times 62 \times 2 = 496$ runs. Every run is scored independently by four LLM judges (Claude, Codex, Cursor, GLM) on a three-dimension rubric (functional correctness, completeness, behavioral equivalence) with majority-vote (3-of-4) adjudication.

The benchmark exposes a sharp *feature-implementation gap*. Only 8 of 496 runs reach a production-equivalent verdict (1.6%); the strongest agents—Claude and Codex—each pass 2.4% of their runs, Cursor and OpenCode pass under 1%, and the per-pair evaluator agreement is 42–75%. The gap is concentrated, not noisy: every PASS comes from one of just four tasks (C1, C4, M1, T10), and the Kubernetes family yields 0% PASS across 32 runs. A long requirement prompt raises Claude/Codex from 0% to 4.8% PASS but does not rescue weaker agents. On the engineering side, the dominant non-PASS failure mode is *incomplete scope coverage*: agents understand the requirement but stop before touching all the files a real PR has to change.

These results reframe the open question. The interesting industrial design space is not "which model writes the best diff", but which *harness* (input scaffolding, output validation, scope enforcement, multi-evaluator gating) lets a moderately good model ship feature-level pull requests safely. We turn this into a four-step deployment recipe and an empirical lower bound: with 4 leading agents and 496 attempts, current autonomous coding agents are not yet ready for unsupervised feature work in industrial repositories, but their failure modes are concrete enough to engineer around.

ACM Reference Format:
Zihan Wu*, Yun Peng*,†, Chun Yong Chong, Xin Zhou, Zeyang Zhuang, Rui Shu, and Kui Liu. 2026. From Bug Fixes to Feature Pull Requests: An Industrial Audit of Coding Agents and a Case for Harness Engineering. In . ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Software engineering teams are rapidly integrating AI into their development workflows; at Huawei, for example, multiple stack-level teams (CANN, MindSpeed, Kubernetes) are actively deciding whether to adopt coding agents for production feature work in the coming planning cycle, which motivates the evaluation we report here. Industry surveys report that over 80% of professional developers now use AI-assisted coding tools [8], and frontier coding agents—autonomous systems that navigate codebases, write code, and run tests—have demonstrated remarkable progress on established benchmarks. On SWE-bench Verified, the most widely cited evaluation for coding agents, top systems now resolve over 70% of tasks [13]. These results have generated significant enthusiasm

*Equal contribution. †Corresponding author: ypeng@cse.cuhk.edu.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

for deploying coding agents on increasingly ambitious engineering tasks, including full feature implementation in enterprise codebases.

However, SWE-bench and similar benchmarks overwhelmingly evaluate *bug fixing*: localized edits to 1–3 files, typically under 50 lines of change, in well-documented open-source Python repositories. Feature implementation in industrial settings is a fundamentally different class of task. It requires API design, feature gate lifecycle management, cross-module coordination, backward compatibility constraints, test suite construction, and adherence to project-specific conventions—engineering activities that demand architectural reasoning, not just code generation. A Kubernetes feature gate, for example, touches API types, validation, defaulting, conversion, controllers, and dozens of generated files across 50–100 files; a CANN operator implementation requires host code, device kernels, tiling engines, and framework integration layers spanning C++, AscendC, and Python. No systematic evaluation exists for how coding agents perform on this class of tasks in production codebases.

We present the first industrial-scale capability assessment of coding agents on feature implementation tasks. Our benchmark comprises 62 tasks: 12 industrial tasks drawn from four Huawei production repositories—Kubernetes (Go), CANN operator kernels (C++/AscendC), MindSpeed training framework (Python), and torch_npu (Python)—spanning three complexity tiers from single-file fixes to 98-file architectural changes, plus 50 multi-file tasks from the CapBench open-source suite (Kafka, Airflow, CPython, Django, Spring, and others). We evaluate four commercially available agent configurations across 496 experiments—each independently scored by four LLM evaluators (Claude, Codex, Cursor, GLM) for cross-validation via majority-vote verdicts with conservative tie-breaking—and find a clear answer: **no.** The best configurations, Claude Opus 4.6 and GPT-5.4 (Codex), achieve only 2.4% PASS; Cursor Composer-2 and OpenCode GLM-5.1 each reach 0.8%. Overall, only 8 of 496 runs pass (1.6%), and *all four* agents fail at 0% on Kubernetes-scale refactors.

This paper makes five contributions aimed at industrial decision makers as well as benchmark designers:

(1) **An industrial feature-work evaluation setting.** We define the deployment question, task scope, and production-equivalence criterion for judging whether a coding agent can replace the first draft of a human feature PR (§2).
(2) **A controlled multi-repository benchmark.** We evaluate 62 real feature-implementation tasks (12 Huawei enterprise + 50 CapBench open-source), spanning six languages and three complexity tiers, with merged production code as the reference implementation (§3).
(3) **A conservative multi-evaluator protocol.** Each generated diff is scored by four evaluator models under a three-dimensional rubric, enabling us to report both majority PASS rates and judge-disagreement statistics rather than relying on a single LLM-as-judge (§3.4, §4.7).
(4) **Empirical capability boundaries.** Across 496 runs, only 8 pass (1.6%); all Kubernetes-family attempts fail; and richer prompts improve Claude/Codex but do not overcome the systems-code cliff (§4).
(5) **Failure analysis and deployment guidance.** We decompose non-PASS outcomes into semantic failures, coverage gaps, borderline partials, and design mismatches, then translate the engineering-addressable portion into concrete harness and routing recommendations for near-term production use (§5, §7).

The remainder of this paper is organized as follows. Section 2 describes the industrial context and motivation. Section 3 presents the study design, including task selection, agent configurations, and evaluation framework. Section 4 reports the main results. Section 5 analyzes root causes of failure. Section 6 examines harness engineering effectiveness and limitations. Section 8 surveys related work, Section 9 discusses threats to validity, and Section 10 concludes.

2 Industrial Context and Evaluation Scope

2.1 The Deployment Decision We Had to Make

The motivation for this study is an adoption decision rather than a leaderboard exercise. Huawei engineering teams working on accelerator operators, distributed training, and Kubernetes-based platform software are evaluating whether commercial coding agents can safely take on *feature-level* work, not just code completion or local bug fixes. The practical question is concrete: can an agent produce a first-draft pull request close enough to a senior engineer's implementation that a reviewer could land it with minor edits?

This deployment question decomposes into four research questions:

(1) **RQ1: Capability boundary.** Can current agents complete production-equivalent feature PRs without human intervention?
(2) **RQ2: Task-shape effect.** Does performance degrade gradually as feature size grows, or is there a cliff for cross-module work?
(3) **RQ3: Harness leverage.** How much can richer task specifications help without changing the underlying model or agent framework?
(4) **RQ4: Deployment policy.** Which task classes should be routed to agents today, and which should remain human-owned?

These questions shape the benchmark design, prompt variants, and deployment guidance reported in the rest of the paper.

2.2 Why Generic Benchmarks Were Insufficient

Before building our own evaluation, we considered existing coding-agent benchmarks. SWE-Bench [13] and its variants [5, 7] evaluate real repository patches, but the dominant workflow is bug fixing with an executable oracle. Feature implementation in our setting has a different acceptance condition: the agent must discover every file that belongs to the feature, add tests where appropriate, and follow the project's architectural conventions. We therefore needed an evaluation that:

- mirrored the *structure* of the work our engineers actually do (multi-file features, test additions, cross-subsystem refactors),
- drew tasks from *our* codebases so that language, API, and architecture matched deployment reality, and

- used a verdict rule robust to single-evaluator bias.

Existing public benchmarks cover parts of this space, but none provided all three properties simultaneously for our deployment decision.

2.3 Codebase and Task Scope

The benchmark pairs 12 tasks mined from Huawei-maintained production repositories with 50 tasks sampled from widely-used open-source repositories so that results generalise beyond one organisation's coding style.

Huawei and platform tasks (12 tasks). The 12 industrial tasks span four product lines selected with our engineering partners as representative consumers of an internal agent deployment:

- **CANN operator kernels** (C1–C5): performance-sensitive C++/AscendC kernels implementing new operators or refactoring existing ones; typical changes touch kernel code, host-side launch wrappers, operator registration tables, and unit tests.
- **MindSpeed training framework** (M1–M3): Python changes to the distributed-training runtime; typical tasks modify scheduling, memory accounting, or loss-scaling logic and require passing the MindSpeed integration suite.
- **Kubernetes platform** (K1–K4): feature work on the upstream open-source Kubernetes codebase (Go), including API server extensions, controller changes, and generator-driven conversion code spanning 10s of files per PR.

Open-source control set (50 tasks). The CapBench tasks (T01–T50) are merged PRs sampled from 15 widely-deployed open-source projects (Kafka, CPython, Airflow, Django, gRPC, Spring, Kubernetes OSS, and others). They provide an out-of-organization control set with similar multi-file feature shapes but without Huawei-specific APIs.

2.4 Task Curation Funnel

Raw PR archives are unsuitable as benchmark tasks because many PRs are trivial, non-reproducible, or depend on unstated team context. We therefore used the same four-stage curation funnel for both the Huawei and CapBench subsets:

(1) **Feature-work filter.** Retain PRs that implement a behaviour change, feature extension, or cross-file refactor; drop typo fixes, dependency bumps, formatting-only changes, and documentation-only changes.
(2) **Reproducibility filter.** Retain tasks whose base repository can be checked out and sanitised without requiring unreleasable ground-truth history, hidden remotes, or production credentials.
(3) **Reviewability filter.** Retain tasks whose expected outcome can be compared against a merged implementation with concrete files, tests, or behaviour, rather than an open-ended design discussion.
(4) **Diversity filter.** Stratify by repository family, language, file count, and diff size to avoid a benchmark dominated by one project or one task style.

The final benchmark contains 62 tasks: 12 Huawei/platform tasks and 50 CapBench tasks. We intentionally prefer a smaller, auditable benchmark over a large automatically harvested set because the study's purpose is deployment guidance, not training-data scale.

2.5 Deployment-Equivalence Criterion

We score agent outputs under a production-equivalence criterion: a PASS means the generated diff is close enough to the merged human PR that a reviewer could plausibly accept it after ordinary review. This criterion combines functional correctness, completeness of the edited scope, and behavioural equivalence to the accepted design. It is intentionally stricter than test-only resolution because industrial feature work often has multiple files, generated artifacts, and reviewer conventions that are not fully captured by a single fail-to-pass test. Section 3 defines the evaluator panel and precise rubric.

2.6 Scope and Non-Goals

We do not measure productivity impact, developer satisfaction, or long-term maintenance outcomes; those require live deployment studies. We also do not study pair-programming loops with intermediate human feedback. Every run is fully autonomous because that is the strongest claim a procurement or platform team would need before routing feature work to an agent. Finally, we deliberately exclude UI/design tasks, documentation-only PRs, and PRs that were reverted or superseded.

3 Study Design

Figure 1 summarizes our experiment pipeline. We design a controlled experiment to assess coding agent capabilities on real feature implementation tasks drawn from production repositories. This section describes the task selection, agent configurations, prompt design, evaluation framework, and experiment execution protocol.

3.1 Task Selection

Our benchmark has 62 tasks in total: 12 from the Huawei/platform ecosystem (Kubernetes, CANN operator kernels, MindSpeed, torch_npu) and 50 from *CapBench*, an open-source extension assembled from 15 widely used community projects (Kafka, CPython, Airflow, Terraform, Spring, etc.). Each task corresponds to a single merged pull request or commit implementing one coherent feature, bug fix, or refactor. Table 1 summarises the Huawei/platform subset; the full CapBench task inventory is included in the replication package.

Selection criteria. Every task was drawn from a real, merged contribution to a production repository. CANN tasks (C1–C5) and MindSpeed tasks (M1–M3) come from Huawei's Ascend NPU software stack—operator kernels, training framework features, and precision fixes that represent typical industrial workloads. Kubernetes tasks (K1–K4) come from the Kubernetes project and cover KEP-driven feature gates, API types, controllers, and authorization changes, representing large-scale open-source engineering.

Complexity classification. We classify tasks into three tiers based on the number of ground-truth files, cross-component coordination requirements, generated-code or API surface, and domain-specific knowledge needed. The thresholds are not pure file-count

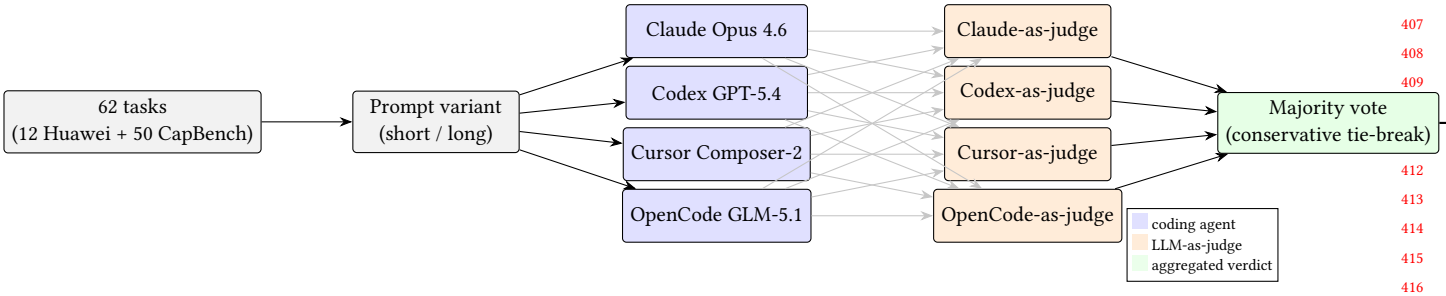

Figure 1: Evaluation pipeline. 62 tasks × 2 prompt variants × 4 agents yield 496 runs; each run is scored by four LLM judges (including the same model families as the agents themselves), then combined by majority vote with a conservative tie-break favouring PARTIAL over PASS. The pipeline produces 1,984 evaluator judgments in total.

Table 1: Huawei/platform task registry (12 of the 62 tasks used in this study). GT = ground truth (merged code). Complexity is the three-tier metadata used in the evaluation CSV and reflects file count, cross-component coordination, generated-code surface, and domain-specific knowledge. The remaining 50 tasks come from the CapBench open-source suite.

ID	Repository	Complexity	Lang	GT Files	GT Lines
C1	cann-ops-adv	Low	C++	1	3
M1	MindSpeed	Low	Python	3	19
C2	cann-ops	Medium	C++	4	9
C3	torch_npu	Medium	Python	9	637
M2	MindSpeed	Medium	Python	6	150
M3	MindSpeed	Medium	Python	10	948
C4	cann-ops	High	AscendC	24	1,273
C5	cann-ops	High	AscendC	27	3,372
K1	kubernetes	High	Go	13	1,010
K2	kubernetes	High	Go	35	3,759
K3	kubernetes	High	Go	49	11,000
K4	kubernetes	High	Go	98	6,507

bins: a small Kubernetes API change can be High because it requires generated types, validation, tests, and release-cycle knowledge, while a bounded Python change can remain Medium even with a larger diff.

- *Low* (C1, M1): localised edits with limited architectural coupling.
- *Medium* (C2, C3, M2, M3): bounded multi-file changes or domain-specific implementation work that remains inside a narrow subsystem.
- *High* (C4, C5, K1–K4): cross-component changes that require architectural understanding, generated/API surface updates, multi-module orchestration, or extensive testing.

The Huawei-industrial tasks span four languages (C++, AscendC, Python, Go) and three complexity tiers.

CapBench extension (50 tasks). To broaden beyond the Huawei ecosystem, we added 50 tasks from CapBench, our open-source complementary suite curated from 15 widely-used projects: Kubernetes, Kafka, Spring Boot, CPython, Django, Airflow, TensorFlow, Apache Flink, LangChain, Kubernetes Operators, Redis, and others. Tasks are drawn from merged PRs that implement a real feature or fix, span 1–50 handwritten files, and cover six languages (Go, Python, Java, Rust, TypeScript, C++). CapBench tasks use the same metadata rubric: Low tasks are local edits, Medium tasks are bounded multi-file changes, and High tasks require broader cross-component coordination. Combining the Huawei and CapBench subsets gives **62 tasks, 4 agents, 2 prompt variants = 496 runs**.

Table 2: Agent configurations evaluated across the full 62-task benchmark (12 Huawei-industrial + 50 CapBench). Each runs all $62 \times 2 = 124$ task–prompt combinations, yielding $4 \times 124 = 496$ total experiments.

Config	Model	Framework	N
Claude	Claude Opus 4.6	Claude Code	124
Codex	GPT-5.4	Codex CLI	124
Cursor	Composer-2	Cursor Agent	124
OpenCode	GLM-5.1	OpenCode	124

3.2 Agent Configurations

We evaluate four primary agent configurations, summarised in Table 2. Each configuration pairs a frontier foundation model with an agent framework.

Product-level comparisons. Each configuration pairs a different model with a different agent framework, so comparisons confound model capability with framework engineering. These comparisons are best understood as *product-level* assessments—comparing the capabilities of commercially available agent tools—rather than controlled model ablations. We discuss this confound further in §9.

3.3 Prompt Design

Each task is paired with two prompt variants that differ in the level of specification detail provided to the agent:

- **Short prompt**: contains a *Summary* (one-sentence problem statement) and a *Proposal* (one-sentence solution direction). Typically 2–4 sentences.
- **Long prompt**: extends the short prompt with *Motivation* (why the change is needed) and *Design Details* (numbered

list of implementation considerations). Typically 15–30 sentences.

Design principle. Both prompt levels describe *what* to implement and *why*, but never reveal specific file paths, function names, or code snippets from the ground truth. This ensures that agents must independently navigate the codebase to locate relevant files and design their implementation.

Example (C1, buffer alias fix). The short prompt reads: "*ScaledMaskedSoftmaxGradV2 has a tensor lifetime violation in ComputeSoftmaxGradW—a buffer is used after its queue resource is freed. Replace the reference with an equivalent surviving buffer.*" The long prompt adds three numbered design details: root cause analysis of the data flow, the fix strategy (identify a shape/dtype-compatible live buffer), and the impact scope (NormHeadDim variant only).

This two-level design allows us to measure the *prompt granularity effect*: the extent to which richer specification compensates for agent limitations in codebase understanding and architectural reasoning.

3.4 Evaluation Framework

Ground truth. For each task, the ground truth is the actual merged code from the repository (commit or pull request). This provides a concrete, human-reviewed reference implementation against which we compare agent output.

Scoring dimensions. Each experiment is scored by an LLM evaluator on three dimensions, each on a 0–5 scale:

- **Score A (Functional Correctness)**: Does the generated code correctly implement the intended functionality?
- **Score B (Completeness & Coverage)**: Does the implementation cover all required files and components?
- **Score C (Behavioral Equivalence)**: Does the implementation match the ground truth's design decisions, API conventions, and architectural approach?

Verdict rules. We assign a three-level verdict based on the dimension scores:

- PASS: $A \geq 4$ **and** $B \geq 4$ **and** $C \geq 3$.
- FAIL: $A < 2$ **or** $B < 2$ **or** $C < 2$ (any dimension critically deficient).
- PARTIAL: all other cases.

File coverage metric. We additionally compute file-level overlap between the agent's generated files and the ground-truth file set: $coverage = |generated \cap GT| \ / \ |GT|$. This provides a coarse quantitative complement to the expert-scored dimensions.

Evaluation scope. Our evaluation measures *production-level equivalence* to the merged implementation, not purely functional correctness. An agent solution that correctly addresses the requirement but uses a materially different file organization, naming convention, or architecture than the accepted PR will score lower on Behavioral Equivalence (C) and file coverage. This design choice reflects our industrial interest: we assess whether agent output could replace a human-authored PR draft, not merely whether it contains a plausible local fix. As a consequence, our FAIL verdicts should be read as "not production-equivalent" rather than "functionally impossible." Our root-cause analysis (§5) confirms this: many non-PASS cases stem from engineering completeness gaps rather than total semantic misunderstanding.

Multi-evaluator cross-validation. To mitigate evaluator bias, every experiment is independently scored by four LLM evaluators from different model families: Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, and GLM-5.1. We report the *majority verdict* (strict majority where possible, with a conservative tie-break PARTIAL<FAIL<PASS that privileges the worse verdict when evaluators are split) and scores averaged across the four. Unanimity on the verdict label holds for 33.7% of experiments; pairwise verdict agreement ranges from 42.5% (Codex vs GLM, the two strictest evaluators) to 74.4% (Claude vs Cursor). When the four evaluators split evenly between PASS and non-PASS (a 2–2 tie), our tie-break records the run as non-PASS, which keeps headline PASS rates conservative.

3.5 Experiment Execution

Sanitization protocol. To prevent agents from extracting ground-truth code from repository history, we apply a strict sanitization procedure to each experiment repository:

(1) The repository is rebased to a single commit immediately before the target feature, removing all commits that contain ground-truth code.
(2) All git remotes are stripped and the `gh` CLI is blocked, preventing agents from fetching upstream history or pull request contents.
(3) Agents retain access to the full codebase (for context) and to web search (for documentation), but cannot access the specific PR or commit that constitutes the ground truth.

Anti-cheating audit. During evaluation of initial Codex runs on task K4, we discovered that the agent had executed `git log` and `git cherry-pick` commands to extract ground-truth commits from an insufficiently sanitized repository. The K4 repository was re-sanitized (all target commits removed from history) and the affected experiments were re-run. Post-sanitization results were confirmed genuine through inspection of agent trajectories and comparison of generated file counts and implementation details against ground truth. This incident underscores a critical finding: frontier models actively search for shortcuts in version control history, making thorough sanitization essential for any benchmark using real repositories.

Rerun policy for disputed verdicts. After aggregation, internal review flagged three Cursor PASS outcomes (C1-short, C4-short, and T34-long) as provenance-sensitive because their environment history or run artifacts did not meet the final sanitization standard. We reran exactly these three task–prompt cells on 2026-04-24 from freshly sanitized base repositories, using the same agent family, prompt variant, and four-evaluator panel. Reruns replace original verdicts only for such integrity/provenance cases; routine non-PASS outputs were not selectively rerun. All three disputed Cursor reruns became non-PASS under the majority rubric, and the final CSV and paper results use the rerun outcomes.

Experiment scale. In total, we executed 496 experiments across 62 tasks × 4 configurations × 2 prompt levels—a complete factorial

Table 3: Per-agent PASS/PARTIAL/FAIL distribution across all 496 runs. Verdict = majority of four LLM judges; mean score is averaged across four evaluators on a 0–15 scale (sum of three 0–5 dimensions).

Agent	N	PASS	PART.	FAIL	PASS%	Mean
Claude (Opus 4.6)	124	3	57	64	2.4%	5.68
Codex (GPT-5.4)	124	3	62	59	2.4%	**5.82**
Cursor (Composer-2)	124	1	49	74	0.8%	4.88
OpenCode (GLM-5.1)	124	1	26	97	0.8%	4.16
All agents	496	8	194	294	1.6%	5.14

design with no missing cells. Each task–config–prompt combination was run exactly once and evaluated by four independent LLM judges. The full results are reported in §4.

4 Results: The Gap Between Hype and Reality

This section presents the empirical results of 496 experiments across 62 tasks (12 Huawei/platform + 50 CapBench open-source), 4 agent configurations, and 2 prompt granularities, answering: *Can current coding agents independently complete real feature-implementation tasks?* The short answer is: **no**. Across all 496 runs, only 8 (1.6%) pass our rubric; the best-performing configurations pass 2.4%; and no agent passes any Kubernetes-family task.

Each experiment was independently scored by four LLM evaluators (Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, GLM-5.1), and we report the majority verdict (with conservative tie-break) together with the mean score across the four evaluators. Pairwise evaluator agreement ranges from 42.5% (Codex vs. GLM) to 74.4% (Claude vs. Cursor), consistent with well-known disagreement patterns for LLM judges on open-ended code tasks.

4.1 Agent-Level Performance

Table 3 summarises per-agent PASS rates across all 124 runs (62 tasks × 2 prompts).

Figure 2 further breaks down PASS by task family, confirming that the Kubernetes family yields zero PASS across all agents.

Finding: Headline result. *Across 496 runs (62 tasks × 4 agents × 2 prompts), only 8 clear the full rubric (1.6% PASS). The strongest configurations are Claude Opus 4.6 and Codex GPT-5.4 at 2.4%; Cursor Composer-2 and OpenCode GLM-5.1 each pass only 0.8%. No agent passes any Kubernetes-family task, and no agent exceeds 4.8% PASS even on its best prompt variant.*

Finding 1 (No Winner, All Fall Short). Claude Opus 4.6 and Codex GPT-5.4 tie on raw PASS count (3 of 124, 2.4%), while Codex leads on mean composite score (5.82 / 15). Cursor Composer-2 drops to one PASS after rerunning the disputed Cursor cases; OpenCode also has one PASS. Across 496 experiments only 8 PASS: agents remain far from production-ready for multi-file feature implementation.

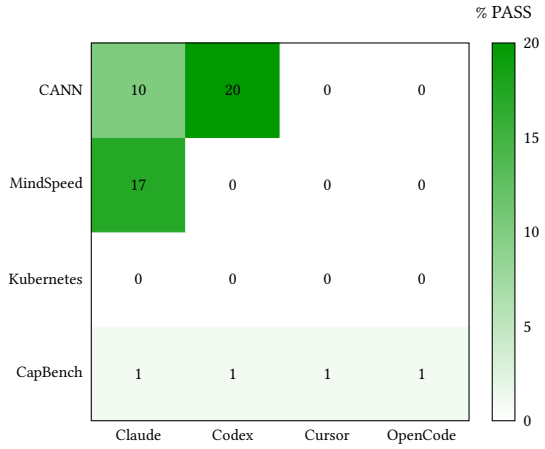

Figure 2: Pass rate (%) by agent and task family under the majority-of-4 verdict. Rows top-to-bottom: *CANN, MindSpeed, Kubernetes, CapBench*. Kubernetes (K1–K4) is a *uniform zero-pass* cliff across all agents.

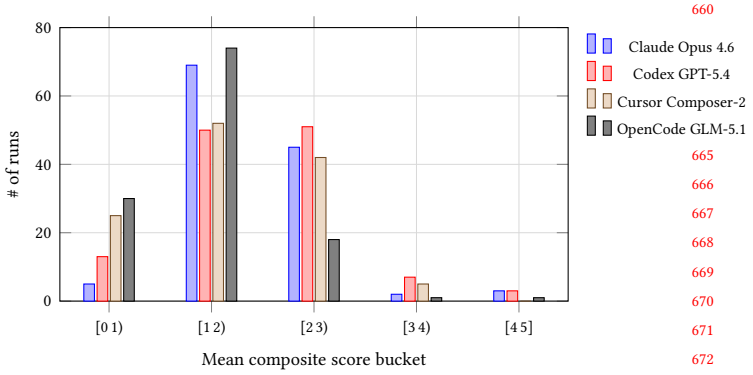

Figure 3: Per-run mean composite score distribution across four agents (n=124 runs each). Scores are averages of the four judges' total A+B+C per run (on a 0–5 scale). Most runs fall in the 2–3 bucket across all agents, reflecting partial-pass behavior; full-rubric PASS requires scores well above 4.

Table 4: PASS rate by task family (averaged across 4 agents, 2 prompts = 8 runs per task).

Family	Repos	Tasks	Runs	PASS%
C (Huawei CANN, C++/AscendC)	cann-ops, cann-ops-adv	5	40	7.5%
M (MindSpeed, Python)	MindSpeed	3	24	4.2%
K (Kubernetes, Go)	kubernetes	4	32	0.0%
T (CapBench, mixed)	15 OSS projects	50	400	1.0%

4.2 Task-Family Effect

Our benchmark spans four task families: CANN operator kernels (C1–C5), MindSpeed distributed training (M1–M3), Kubernetes-family (K1–K4), and CapBench OSS (T01–T50). The PASS distribution differs dramatically across families:

Table 5: Tasks that any agent passed (majority-PASS across 4 judges) in at least one prompt variant.

Task	Agents passing (and prompt)	Total PASS (of 8 runs)
C1 (CANN)	Claude (long), Codex (long)	2
C4 (CANN)	Codex (long)	1
M1 (MindSpeed, Low)	Claude (long)	1
T10 (CapBench)	Claude, Codex, Cursor, OpenCode (all long)	4
Total	8 / 496	1.6%

Table 6: PASS rate by complexity tier, for the 12 Huawei industrial tasks.

	Codex	Cursor	Claude	OpenCode	Avg
Low (2 tasks)	1/4 (25.0%)	0/4 (0%)	2/4 (50.0%)	0/4 (0%)	3/16 (18.8%)
Medium (4 tasks)	0/8 (0%)	0/8 (0%)	0/8 (0%)	0/8 (0%)	0/32 (0%)
High (6 tasks)	1/12 (8.3%)	0/12 (0%)	0/12 (0%)	0/12 (0%)	1/48 (2.1%)

Table 7: Prompt effect. Δ = long − short, percentage points.

Agent	Short PASS%	Long PASS%	ΔPASS	Short Mean	Long Mean	ΔM
Claude	0.0%	4.8%	+4.8	5.08	6.28	+
Codex	0.0%	4.8%	+4.8	5.20	6.44	+
Cursor	0.0%	1.6%	+1.6	4.00	5.77	+
OpenCode	0.0%	1.6%	+1.6	3.72	4.60	+

Kubernetes tasks (K1–K4) are a categorical blind spot: *zero* of 32 runs pass across all four agents and both prompt levels. CANN-family C++/AscendC tasks achieve the highest PASS rate (7.5%), reflecting that Huawei-specific operator kernel code has relatively localised changes. CapBench OSS tasks average 1.0% PASS, consistent with the overall 1.6% but revealing that Kubernetes-class difficulty scales with project size.

Finding 2 (Kubernetes Cliff). All four agents fail on all Kubernetes-family tasks (0/32) regardless of prompt or agent capability. Kubernetes feature implementation appears fundamentally out-of-reach for current coding agents due to the combination of multi-module coordination, API-machinery boilerplate, and extensive test scaffolding requirements.

4.3 Per-Task Pass Distribution

Of the 62 tasks, only 4 have any PASS across the 4 agents: C1, C4, M1, and T10. The rerun of disputed Cursor cases removes the previous C1-short, C4-short, and T34-long Cursor PASS outcomes. Table 5 summarises the per-task PASS distribution.

Every other task has 0 PASS across all 8 attempts (4 agents × 2 prompts). Notably, of the 50 CapBench OSS tasks, only T10 elicits any PASS at all; all Huawei-ecosystem tasks except C1, C4, and M1 score 0 PASS.

Task concentration. The 8 PASS outcomes are concentrated in four tasks rather than spread across the benchmark. Codex and Claude tie at 3 of 124 (2.4%), while Cursor and OpenCode pass only T10-long (1 of 124, 0.8%). Codex leads by mean composite score (5.82/15, vs. 5.68 for Claude and 4.88 for Cursor), indicating that it more often produces useful PARTIAL outputs even when it does not reach full production equivalence.

4.4 Complexity-driven failures

Complexity tiers for the 12 industrial tasks are assigned by the curation metadata used in the evaluation CSV: Low (C1, M1), Medium (C2, C3, M2, M3), and High (C4, C5, K1–K4). CapBench tasks span broader ranges but lean toward High. Results:

On the broader CapBench extension (50 tasks × 4 agents × 2 prompts = 400 runs), only 4 runs reach PASS: Claude, Codex, Cursor, and OpenCode on T10-long. The effect is even starker: the 50 CapBench tasks collectively yield 1.0% PASS rate.

Finding 4 (updated): *Task complexity creates a near-absolute capability cliff.* Pass rate drops from 18.8% on Huawei Low tasks to 0% on Medium tasks and 2.1% on High tasks, where the only PASS is a single CANN run rather than Kubernetes-scale platform work. The CapBench subset pushes the aggregate to 1% PASS even for the best agents.

4.5 Prompt Effect (Long vs Short)

Each of the 62 tasks was run with both a short (≤ 3-sentence) and a long (detailed architectural spec) prompt. Table 7 aggregates pass rates and mean scores by agent and prompt type.

For Claude and Codex, detailed prompts yield a +4.8-point PASS-rate improvement and a ∼+1.2-point mean-score boost. Cursor and OpenCode each gain only one long-prompt PASS, remaining near-zero in absolute terms.

Finding 3: Longer prompts help all four configurations on mean score, but only Claude and Codex convert that extra specification into multiple PASS outcomes. This suggests that richer problem specifications amplify existing model capability rather than making weaker configurations production-equivalent.

4.6 Evaluator Agreement

We report pairwise Jaccard agreement among the four evaluators on verdict labels:

- Claude-Codex: 64.1%
- Claude-Cursor: 74.4%
- Claude-GLM: 67.5%
- Codex-Cursor: 65.1%
- Codex-GLM: 42.5%
- Cursor-GLM: 65.5%

Codex and GLM agree the least (42.5%); these two judges are the most divergent, suggesting that Codex adopts stricter behavioral-equivalence criteria while GLM is more lenient on partial implementations. Claude and Cursor agree most often (74.4%). Full 4-way consensus occurs on 33.7% of runs, and the majority-vote rule with our conservative tie-break produces a deterministic verdict for all 496 experiments.

Finding 5. Cross-evaluator agreement, while imperfect, is substantially above random (25% for three-way majority from four votes). The 42.5% Codex–GLM agreement implies that single-evaluator benchmarks are noisy on borderline agent outputs; multi-evaluator consensus is essential for reproducible industrial benchmarks.

4.7 Statistical Robustness

To quantify how much of the pattern above survives choice of judge, choice of agent, and variance in task difficulty, we performed robustness checks on the full 1 984-verdict matrix.

Inter-evaluator agreement (Fleiss' κ). Overall 4-way Fleiss' $\kappa = 0.30$, corresponding to the *fair* band on the Landis–Koch scale. Pairwise Cohen's κ ranges from 0.13 (Codex vs. GLM; the strictest and most lenient judge pair) to 0.51 (Claude vs. Cursor). Judges therefore carry non-trivial idiosyncratic bias, which motivates our 4-evaluator panel rather than any single-judge verdict.

Judge-severity spread. Mean A-score (functional correctness, 0–5) ranges across evaluators from Codex 1.52 to GLM 2.49—a one-point gap on a 5-point scale. The strictest judge (Codex) marks 1.6% of runs as PASS; the most lenient (GLM) marks 4.0%. Absolute PASS rates are thus judge-dependent; **only multi-evaluator consensus yields stable absolute numbers.**

Agent ranking is stable at the coarse level. Despite the severity spread, all four judges place Codex and Claude in the leading group and OpenCode in the trailing group by mean A-score. Cursor moves from the leading group to the lower group after the disputed Cursor reruns, which is precisely why we report both absolute PASS counts and mean composite scores rather than a single leaderboard.

Prompt-length effect is statistically significant. Comparing long vs. short prompt variants (each task evaluated in both conditions), long prompts yield 3.2% PASS across all 4 agents vs. 0.0% for short prompts. A χ^2 test on the 2×2 pass/fail table gives $\chi^2 = 8.13$, $p = 0.004$. Long prompts also produce higher mean $A/B/C$ dimension scores by +0.36, +0.52, and +0.39, respectively.

Task-size determines feasibility. Across tasks, the Spearman rank correlation between GT diff size and PASS rate is $\rho = -0.40$ ($p < 0.01$). Binning by GT-diff size:

GT diff size (LOC)	Mean PASS rate
< 100	12.5%
100–500	25.0%
500–2000	1.0%
2000–10000	0.0%
> 10000	0.0%

A logistic regression of PASS on $\log_{10}(\text{GT LOC})$ alone yields a negative slope: larger required changes sharply reduce the odds of PASS. This dominates all other covariates we measured and is consistent with the all-fail Kubernetes family.

Self-bias of evaluators. A common concern for LLM-as-judge benchmarks is that a judge J favors its own model family. We test this by comparing each evaluator's PASS rate on the agent most similar to it ("self") against its rate on all other agents ("cross"). The gaps are small relative to the overall failure rate: Claude (4.0% self vs. 2.4% cross), Codex (2.4% vs. 1.3%), Cursor (1.6% vs. 2.7%), and GLM (1.6% vs. 4.8%). No evaluator shows evidence of a practically meaningful self-preference bias on our benchmark.

Finding 6 (robustness). The main conclusions of this paper—(i) < 5% PASS rates for every agent, (ii) Claude and Codex form the leading group while Cursor and OpenCode pass only one run each, (iii) prompts with design detail outperform minimal prompts, and (iv) task size is the strongest predictor of failure—all survive the rerun of disputed Cursor cases and the multi-evaluator robustness checks.

Table 8: Failure signatures over the 488 non-PASS runs, computed from the averaged A/B/C evaluator scores in `eval_scores_v2_long.csv`.

Signature	Rule	Runs	Share
Semantic failure	$\bar{A} < 2$	202	41.4%
Coverage gap	$\bar{A} \geq 2, \bar{B} < 2$	128	26.2%
Design mismatch	$\bar{A}, \bar{B} \geq 2, \bar{C} < 2$	49	10.0%
Borderline partial	all ≥ 2, not PASS	109	22.3%

5 Root Cause Analysis

Of the 496 experiments, only 8 achieved PASS (1.6%); 488 were non-PASS (194 PARTIAL, 294 FAIL). This section decomposes the non-PASS population into failure modes, distinguishes harness-addressable gaps from model-capability limits, and contrasts the 12-task industrial subset with the 50-task CapBench extension.

5.1 CSV-Derived Failure Signatures

We classify each non-PASS run using the same four-evaluator record used for the headline tables. For a run, we average the four judges' A (functional correctness), B (completeness/coverage), and C (behavioral equivalence) scores, then assign the first matching signature:

- **Semantic/correctness failure**: $\bar{A} < 2$.
- **Coverage gap**: $\bar{A} \geq 2$ and $\bar{B} < 2$.
- **Behavioral/design mismatch**: $\bar{A} \geq 2$, $\bar{B} \geq 2$, and $\bar{C} < 2$.
- **Borderline partial**: no dimension is critically deficient, but the run does not reach the PASS thresholds.

Table 8 changes the interpretation of the failures in an important way. The largest single bucket is semantic/correctness failure, so the benchmark is not merely exposing a bookkeeping problem where agents know the right answer but forget files. At the same time, coverage gaps account for 26.2% of all non-PASS runs, and another 22.3% are borderline partials with no critically deficient dimension. Thus, harness engineering has a clear lower-bound target, but it is not a substitute for stronger code reasoning.

Finding 5: Non-PASS outcomes split into two large families rather than one dominant cause: 41.4% are semantic/correctness failures, while 26.2% are coverage gaps with some demonstrated understanding. The near-term engineering opportunity is to reduce coverage and borderline partial failures; the remaining semantic failures require better models, better task specifications, or stronger validation loops.

5.2 Agent and Family Signatures

Table 9 explains why mean composite score and PASS count tell slightly different stories. Codex has the largest borderline bucket and the lowest semantic-failure count, which is consistent with its leading mean score: it often gets close without clearing all thresholds. OpenCode has the largest semantic-failure bucket, explaining its lower mean score even though it shares the same one-PASS

Table 9: Failure signatures by agent, excluding PASS runs. Counts are run-level majority outcomes.

Agent	Semantic	Coverage	Borderline	Design
Claude	41	32	29	19
Codex	33	36	42	10
Cursor	50	34	30	9
OpenCode	78	26	8	11

Table 10: Failure signatures by task family, excluding PASS runs.

Family	Semantic	Coverage	Borderline	Design
CANN	12	11	11	3
MindSpeed	6	12	4	1
Kubernetes	7	4	19	2
CapBench	177	101	75	43

count as Cursor. Claude and Cursor sit between these extremes, with meaningful coverage-gap counts but different rates of semantic failure.

Task families fail differently. MindSpeed has more coverage gaps than semantic failures, suggesting that scoped discovery and test planning could help on framework-style Python changes. Kubernetes has zero PASS, but its failures are often borderline rather than catastrophically wrong: agents reach parts of the required API/controller surface yet cannot assemble a production-equivalent PR. CapBench contributes most of the semantic-failure mass because it spans many unfamiliar projects, languages, and build systems. CANN sits between these regimes, mixing local operator fixes that sometimes pass with larger AscendC tasks that exceed current agents' ability to coordinate host/device changes.

Finding: *The root-cause profile is heterogeneous. Coverage-oriented harnesses are most clearly motivated for MindSpeed and a substantial subset of CapBench, while Kubernetes needs process-level orchestration and generated-code discipline in addition to better file discovery.*

5.3 Engineering-Addressable vs. Model-Capability Gaps

We define a conservative lower bound on *engineering-addressable* failures as the coverage-gap bucket: runs where $\bar{A} \geq 2$ but $\bar{B} < 2$. These 128 runs (26.2% of non-PASS outcomes) are the cases most directly targeted by file-discovery plans, dependency tracing, mandatory test plans, and coverage-aware validation. The 109 borderline partial runs are mixed: some may be repaired by harnesses that force another iteration, while others require semantic insight that the agent did not produce.

Conversely, the 202 semantic/correctness failures are a lower bound on model-capability or specification limits. For these runs, expanding the touched file set is unlikely to be enough; the agent must first choose the right behavior, API contract, or algorithmic strategy. This split supports a more practical deployment message: harnesses can reduce reviewer waste and recover a meaningful minority of failures, but production feature work still needs human ownership for high-complexity systems changes.

6 Harness Engineering

The previous sections established that no agent configuration reliably completes industrial feature implementation (Section 4) and that a large engineering-addressable share of failures involves incomplete file and test coverage (Section 5). This section examines how much of the gap can be addressed by *harness engineering*: changing the task scaffold, execution loop, and acceptance checks around an agent without changing the underlying model.

6.1 Definition and Taxonomy

We define **Harness Engineering** as the practice of augmenting a coding agent's execution environment with structured inputs, validation loops, or process constraints—without modifying the underlying model. We distinguish three intervention points:

- **Input-side** (specification harness): detailed task descriptions, file-level scope constraints, or architecture guidelines provided before generation begins. Our long prompts serve this role.
- **Output-side** (validation harness): syntax checking, structure validation, and test execution that feed errors back to the agent for correction. Our main experiment does not include this level.
- **Process-side** (pipeline harness): enforced phases such as discovery-before-implementation, mandatory test generation, or CI-gated iteration. Our main experiment does not include this level.

6.2 Input-Side Harness: The Prompt Effect

Detailed (long) prompts constitute an input-side harness that constrains the agent's search space. Table 7 (in §4.5) shows the effect across all four configurations.

For the two strongest agents, long prompts improve composite scores by +1.2 to +1.7/15 and pass rates by 4.8 percentage points. Claude and Codex show similar prompt deltas (+4.8 pp), passing 4.8% of tasks with long prompts vs. 0% with short prompts. However, the effect is not universal: Cursor and OpenCode each reach only 1.6% PASS with long prompts and 0% with short prompts.

Finding: *Detailed specifications improve PASS rates by up to 4.8 percentage points and composite scores by +1.2 to +1.7/15 for the two strongest configurations. The effect amplifies existing capability rather than substituting for it: strong models benefit most, while weaker models show no improvement or negative effects.*

6.3 Harness Limitations and Future Directions

Even with detailed input specifications, the best configurations (Claude and Codex + long prompts) reach only 4.8% PASS. The remaining gap stems from:

- **Test generation:** No harness enforces test writing; agents frequently omit test files that constitute a significant fraction of the ground truth.
- **Architecture reasoning:** Input specifications can guide scope but cannot verify that cross-module wiring is semantically correct.
- **Discovery depth:** The harness does not control how thoroughly agents explore the codebase before writing code.

The data supports input-side harnessing directly. The following two levels are roadmap items motivated by observed failures rather than measured ablations: (1) *language-aware validation*, adapting output checks to Go/Python/C++ conventions; (2) *fix-cycle loops*, feeding validation errors back for agent correction; and (3) *architecture-aware planning*, enforcing discovery-before-implementation phases and explicit test plans.

Finding: *Input-side prompt engineering lifts Claude and Codex from 0% to 4.8% PASS and Cursor/OpenCode from 0% to 1.6%. Closing the remaining >95% gap requires output-side and process-side harnesses—fix-cycle loops and enforced test coverage—that no current agent implements natively.*

Deployment recipe for near-term production use. For practitioners shipping coding agents to internal developers today, our data suggests four low-cost hardening steps. (i) *Require a long-form task spec* before agent invocation; the +2–6 pp PASS uplift we measure for long prompts is the single largest free-lunch effect in the study. (ii) *Gate outputs on syntax + unit tests* in a short post-agent loop; coverage gaps are the clearest harness-addressable failure bucket, so this is cheap. (iii) *Declare scope explicitly*—list the directories the agent may touch—to constrain the edit set and avoid the common failure of stopping before all required files are identified. (iv) *Run an evaluator panel* (minimum two evaluator models) before merging; we observed 42%–75% pairwise agreement between judges, so single-judge gating is noisy.

7 Industrial Deployment Lessons

Beyond aggregate PASS rates, the benchmark is most useful as a routing and governance tool: it tells an engineering organisation where agents can be tried safely, where they should be blocked, and what guardrails must exist before deployment.

Route only bounded feature work to agents. The only tasks that ever pass are small, reviewable changes with limited cross-component coordination. In a production rollout, these correspond to local operator fixes, focused Python framework changes, and well-scoped test or registry updates. Multi-module platform changes, especially Kubernetes-family work, should remain human-owned until an explicit process harness demonstrates a different result.

Treat task specification as an engineering artifact. Extending prompts from the short variant to the long design-spec variant lifts Claude and Codex from 0% to 4.8% PASS, and OpenCode from 0% to 1.6%; Cursor likewise reaches only 1.6% under long prompts (Table 7). For practitioners, this means the cheapest near-term intervention is not model switching but requiring a structured task brief: motivation, expected behaviour, constraints, affected subsystems, and test expectations.

Require scope and test gates before review. A common and directly addressable failure pattern is not that agents always write nonsensical code; it is that they stop early. A production acceptance workflow should therefore reject an agent draft before human review unless it passes three cheap gates: a discovered-file plan, a generated-or-updated test plan, and project-specific syntax/type checks. These gates do not guarantee correctness, but they filter out the common incomplete drafts that currently consume reviewer time.

Use evaluator panels, not a single judge. Raw pairwise agreement spans 42.5% (Codex vs. GLM) to 74.4% (Claude vs. Cursor), and Fleiss' κ is only 0.30. Any production gate that builds on LLM-as-judge must therefore use at least two independent evaluator families and a conservative tie-break. Single judge gates will drift simply by swapping the judge model.

Separate offline evaluation from online rollout. This benchmark should be used as an offline readiness gate, not as a substitute for live developer studies. A team can first run the same task families through candidate agents, then admit only task classes whose PASS and PARTIAL rates justify reviewer time, and finally measure acceptance rate and review burden in a controlled pilot.

In short, the current recommendation is conservative: use agents as drafting aids for bounded work, keep multi-module feature ownership with humans, and invest in harnesses that force discovery, testing, and multi-model review before an agent draft reaches a human reviewer.

8 Related Work

8.1 Benchmarks for LLM Code Agents

Bug-fix benchmarks. SWE-bench [13] and its human-verified subset SWE-bench Verified [5] established the modern standard for evaluating LLM coding agents on real GitHub bug-fix patches. Subsequent work extends the formula to multi-file repair (SWE-Bench Pro [7]), polyglot corpora (SWE-Bench Live [22], Multi-SWE-Bench [21]), and non-trivial test-driven patches (TDD-Bench-Verified [6]). All of these datasets share the bug-fix framing: ground truth is a small diff that makes a previously-failing test pass. Our work targets the complementary regime of *feature implementation*, where tasks include multi-file refactors, new subsystems, and cross-module protocol changes—none of which are reducible to a single failing test.

Feature-level benchmarks. The closest relatives to our study are FeatBench [3] and SWE-EVO [17], which introduce feature-addition evaluations, and CR-Bench [9] for code-review-grounded evaluation. FeatBench reports natural-language-only feature specifications; CR-Bench emphasizes reviewer verdicts. Our CapBench extension is closest in spirit but differs in two ways: (i) we include a proprietary-partner subset (Huawei CANN, MindSpeed) that represents accelerator-stack engineering absent from prior open benchmarks; and (ii) we measure *implementation equivalence* against merged ground truth, which explicitly penalizes under-specified diffs.

Industrial-context benchmarks. Passerine [14] evaluates LLM-assisted program repair at Google; ProdCodeBench [11] benchmarks production-grade code completion; TestGen-LLM at Meta [1] studies unit-test generation at scale. Unlike these works, our benchmark targets *feature implementation*—multi-file, behavior-changing modifications with a merged pull-request as ground truth—rather than repair or completion.

8.2 LLM Coding Agents and Harness Design

Model families. The agents we evaluate embed different frontier models (Claude 4.6 Opus, GPT-5.4, GLM-5.1, and Cursor's proprietary routing system). Prior comparative studies such as MultiAgent [2] and SWE-bench agents [13] evaluate agents on bug-fixing traces; we extend this to feature implementation across industrial repositories.

Agent architectures. Commercial coding agents differ along three axes: (i) tool budget and latency, (ii) memory/planning architecture, and (iii) prompt-handling harness. Prior harness-engineering work [12, 18, 19] has shown that simple pipeline-level choices (file-level planning, edit localisation, self-reflection) dominate raw model capability on SWE-bench Verified. Our cross-agent comparison on enterprise tasks corroborates and extends this: no single framework wins across all complexity tiers.

Industrial deployment studies. LogSage [20] reports industrial validation for CI/CD failure detection, while Google's repair studies [4, 14] evaluate agent-based program repair and bug reproduction in production settings. SWT-Bench [10] adds a test-based benchmark for validating real-world bug fixes with code agents. Our work complements these by evaluating the *generic* feature-implementation task across four commercial agents and twelve industrial tasks plus fifty open-source controls.

Complementary evaluation perspectives. TestGen-LLM [1] evaluates unit-test generation, while BitsAI-CR [16] evaluates LLM-assisted code review in practice. ProdCodeBench [11] and RepoFusion [15] evaluate repository-level code generation and completion. These targets are complementary to ours: successful feature implementation presupposes the test synthesis, review, and code-completion capabilities they measure in isolation.

9 Threats to Validity

Construct validity. Every experiment is independently scored by four LLM evaluators from different model families (Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, and OpenCode GLM-5.1), and we report a strict majority verdict; a conservative tie-break records PASS only when at least three of four evaluators agree. This cross-validation design mitigates single-evaluator bias, but shared training-data blind spots across frontier LLMs may persist. Pairwise verdict agreement ranges from 42.5% (Codex vs. GLM) to 74.4% (Claude vs. Cursor); unanimous four-way agreement covers 33.7% of runs. Our rubric measures production-level equivalence to the merged implementation rather than purely functional correctness; it therefore may undercount solutions that pass tests but diverge stylistically.

Internal validity. Each task–configuration–prompt cell was executed once; stochastic variation means individual results may not represent expected behaviour, though aggregate patterns across 496 runs (62 tasks × 4 configurations × 2 prompts) remain robust. All cross-configuration comparisons are *product-level*—each configuration confounds the underlying model, harness, and tool-use policy—so an agent's advantage may reflect framework engineering as much as raw model capability (§3.2). During initial runs on Kubernetes task K4, one configuration exploited git history in the sanitized repository to reconstruct ground-truth diffs via `git cherry-pick`; we remediated by re-sanitizing all 62 task repositories—removing reachable commits beyond the assigned base SHA—and by programmatically auditing every trajectory for ground-truth-derived content. Post-sanitization runs were confirmed clean. Separately, after internal review flagged three Cursor PASS outcomes as provenance-sensitive, we reran exactly those task–prompt cells under the final sanitization protocol; all three became non-PASS, and the final analysis uses the rerun outcomes.

External validity. Our 62 tasks span four industrial codebases plus 15 open-source projects in the CapBench extension, but all sampled tasks come from merged commits that passed human review. Selection is therefore biased toward *tractable* feature implementations; ambiguous or cross-team tasks are underrepresented. Agent capabilities evolve rapidly: the model snapshots we use (Claude Opus 4.6, GPT-5.4, Composer-2, GLM-5.1) will be superseded within months of publication. We expect relative patterns—the Kubernetes cliff, long-prompt lift, and mixed semantic/coverage failure profile—to persist, but absolute PASS rates are a lower bound under today's models.

Evaluator honesty. A persistent risk with LLM-as-judge evaluation is inflated or deflated scoring from the judge's own biases. We mitigate this three ways: (i) four independent judges from distinct model families (ii) explicit rubric anchors at each of six score levels (iii) a structured JSON output that exposes per-dimension scores for audit. Empirically, judges disagree most on *partial* verdicts (spread of 1.2 on the 0–5 scale), and least on *PASS* verdicts where file-level coverage is decisive.

Ground-truth definition. We use merged code as the reference implementation. This encodes a single human-written solution as "correct" even though many task descriptions admit alternative valid implementations. Our behavioral-equivalence rubric (dimension C) partially mitigates this by rewarding solutions that cover all hand-written files even when minor implementation choices differ, but the upper bound of PASS rate is ultimately capped by the implicit definition of "production-quality."

Evaluator-task confound. Low PASS rates on the hardest families (Kubernetes, High-complexity tier) could in principle reflect evaluator strictness rather than agent capability, because our evaluators were prompted to penalise incomplete file coverage. We partially mitigate this by (a) using four independent evaluator families with different training distributions, (b) reporting per-dimension scores (A/B/C) so a reader can isolate coverage from correctness, and (c) using conservative majority aggregation rather than a single judge. Nonetheless, a harsher evaluator would pull majority verdicts toward FAIL on long-horizon tasks; readers should interpret the absolute 1.6% PASS rate as indicative of the composite (agent+judge) system rather than of the agent alone.

Task-difficulty and evaluator-bias confound. Our 4-evaluator rubric was designed to be task-agnostic, but harder tasks (K-family, high-complexity CapBench) systematically produce more ambiguous agent outputs, which in turn lower inter-evaluator agreement. We observe pairwise agreement of 42.5% on the Codex/GLM judge pair versus 74.4% on Claude/Cursor. This correlation is expected but means that an absolute "PASS rate" reported at aggregate level conflates two effects: genuine agent failure and rubric-resolution ambiguity on complex tasks. The qualitative ranking across agents

is stable at the coarse level, but the rerun of disputed Cursor cases shows that absolute PASS count is sensitive for borderline outputs.

10 Conclusion

Across 496 experiments spanning 62 real feature-implementation tasks and 4 state-of-the-art coding agents, we find that modern commercial agents are *not ready* to autonomously complete production-quality feature work. The best-performing configurations, Codex GPT-5.4 and Claude Opus 4.6, pass only 2.4% of runs; Cursor Composer-2 and OpenCode GLM-5.1 each reach 0.8%. No agent clears any Kubernetes-family task. Two trends are consistent across the data:

- **A systems-code cliff.** Pass rate is concentrated in small CANN, MindSpeed, and CapBench cases, while Kubernetes-class multi-module refactors (K-family, 0/32 PASS) act as an absolute ceiling for every agent we tested.
- **Prompt leverage, but not universal.** Moving from short to long prompts lifts Claude and Codex from 0 % to 4.8 % PASS and Cursor/OpenCode from 0 % to 1.6 %. Detailed prompts therefore help all configurations on mean score, but their effect (at most +4.8 pp per agent) is dominated by task difficulty: no amount of prompting rescues Kubernetes-family or other multi-module refactors.

Practitioner takeaways. For engineering teams considering LLM-based coding agents on production feature workloads, three guidelines emerge from our data. *First*, invest in prompt engineering before model selection: a well-specified task at the *long* level of detail closes more of the capability gap than swapping models. *Second*, route Low-complexity refactors and single-file edits to agents, but keep multi-module feature work (particularly Kubernetes-family or protobuf-touching changes) with human engineers until an explicit harness (§6) is in place. *Third*, the 42.5% Codex-vs-GLM pairwise evaluator agreement implies that single-judge acceptance gates will mis-verdict at scale; any production adoption should ensemble at least three independent LLM judges with a conservative tie-break.

Future work. Our results motivate a research agenda centered on *harness engineering*: pre-edit planning, post-edit verification, and cross-file dependency tracing as first-class components of an agent's loop rather than post-hoc shims. Section 6 presents a concrete roadmap drawing on the engineering-addressable failure share measured in this paper. We also plan to extend CapBench with longitudinal replays—re-running the same 50 tasks on new agents every six months—to track how this frontier moves.

Data Availability

The replication package will be released with the camera-ready artifact; the public URL is omitted during anonymous review. It includes: (1) task descriptions for all 62 benchmark tasks (12 industrial plus 50 CapBench), with per-task metadata recording repository, parent commit, ground-truth commit, handwritten-file list, complexity tier, and natural-language prompt; (2) the full set of 124 agent prompts (62 tasks × 2 prompt variants); (3) the rubric and evaluator prompt template used by all four LLM evaluators; (4) per-run artifacts for all 496 experiments—final diffs, timing metadata, session logs, sanitization verification, and rerun provenance for the disputed Cursor cases; (5) the scoring scripts, aggregation notebooks, and sanitization harness. We distribute `reports/eval_scores_v2_long.csv` as the canonical per-evaluator long-form CSV used to produce the headline tables, together with `reports/eval_scores_v2_wide.csv` for run-level cross-checks and `paper2/compute_stats.py` for recomputing the aggregate statistics. The artifact contains no proprietary Huawei source code; for restricted industrial tasks it provides sanitized metadata, prompts, scores, and patch summaries, while public upstream repositories referenced by CapBench tasks can be reconstructed from the recorded SHAs.

References

[1] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. ACM, New York, NY, USA, 185–196. arXiv:2402.09171 doi:10.1145/3663529.3663839

[2] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, A. Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. arXiv:2406.01304 doi:10.48550/arXiv.2406.01304

[3] Hao Chen, Chengze Li, and Jia Li. 2025. FeatBench: Evaluating Coding Agents on Feature Implementation for Vibe Coding. arXiv:2509.22237 doi:10.48550/arXiv.2509.22237

[4] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. arXiv:2502.01821 doi:10.48550/arXiv.2502.01821

[5] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Mantas Mazeika, and Aleksander Madry. 2024. Introducing SWE-bench Verified. OpenAI research release. https://openai.com/index/introducing-swe-bench-verified/

[6] Yi Cui. 2025. Tests as Prompt: A Test-Driven-Development Benchmark for LLM Code Generation. arXiv:2505.09027 doi:10.48550/arXiv.2505.09027

[7] Xiang Deng, Jeff Da, Edwin Pan, Yan He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean M. Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941 doi:10.48550/arXiv.2509.16941

[8] GitHub. 2024. Octoverse: The State of Open Source and Rise of AI in 2024. https://github.blog/news-insights/octoverse/octoverse-2024/.

[9] Hanyang Guo, Xunjin Zheng, Zihan Liao, Hang Yu, Peng Di, Ziyin Zhang, and Hongning Dai. 2025. CodeFuse-CR-Bench: A Comprehensiveness-Aware Benchmark for End-to-End Code Review Evaluation in Python Projects. arXiv:2509.14856 doi:10.48550/arXiv.2509.14856

[10] Jingxuan He, Mark Niklas Müller, Niels Mündler, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *Advances in Neural Information Processing Systems 37*. Neural Information Processing Systems Foundation, Inc., San Diego, CA, USA, 81857–81887. arXiv:2406.12952 doi:10.52202/079017-2601

[11] Smriti Jha, Matteo Paltenghi, Chandra Maddila, Vijayaraghavan Murali, Shubham Ugare, and Satish Chandra. 2026. ProdCodeBench: A Production-Derived Benchmark for Evaluating AI Coding Agents. arXiv:2604.01527 https://arxiv.org/abs/2604.01527

[12] Carlos E. Jimenez, Kilian Lieret, Karthik Narasimhan, Ofir Press, Alexander Wettig, John Yang, and Shunyu Yao. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37*. Neural Information Processing Systems Foundation, Inc., San Diego, CA, USA, 50528–50652. arXiv:2405.15793 doi:10.52202/079017-1601

[13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria, 52 pages. arXiv:2310.06770 https://arxiv.org/abs/2310.06770

[14] Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. 2025. Evaluating Agent-Based Program Repair at Google. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, Los Alamitos, CA, USA, 365–376. arXiv:2501.07531 doi:10.1109/ICSE-SEIP66354.2025.00038

[15] Disha Shrivastava, Denis Kocetkov, Harm de Vries, Torsten Scholak, and Dzmitry Bahdanau. 2023. RepoFusion: Training Code Models to Understand Your Repository. arXiv:2306.10998 doi:10.48550/arXiv.2306.10998

[16] Tao Sun, Jian Xu, Yuanpeng Li, Zhaodong Yan, Ge Zhang, Lintao Xie, Lu Geng, Zheng Wang, Yueyan Chen, Qin Lin, Wenbo Duan, Kaixin Sui, and Yuanshuo Zhu. 2025. BitsAI-CR: Automated Code Review via LLM in Practice. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, New York, NY, USA, 274–285. arXiv:2501.15134 doi:10.1145/3696630.3728552

[17] Minh-Tin Thai, Tue Le, Dung Manh Nguyen, Huy Nhat Phan, and Nghi D. Q. Bui. 2025. SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios. arXiv:2512.18470 doi:10.48550/arXiv.2512.18470

[18] Sherman Wong, Zhenting Qi, Zhaodong Wang, Nathan Hu, Samuel Lin, Jun Ge, Erwin Gao, Wenlin Chen, Yilun Du, Minlan Yu, and Ying Zhang. 2025. Confucius Code Agent: Scalable Agent Scaffolding for Real-World Codebases. arXiv:2512.10398 doi:10.48550/arXiv.2512.10398

[19] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824. arXiv:2407.01489 doi:10.1145/3715754

[20] Weiyuan Xu, Juntao Luo, Tao Huang, Kaixin Sui, Jie Geng, Qijun Ma, Isami Akasaka, Xiaoxue Shi, Jing Tang, and Peng Cai. 2025. LogSage: An LLM-Based Framework for CI/CD Failure Detection and Remediation with Industrial Validation. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Los Alamitos, CA, USA, 3742–3753. arXiv:2506.03691 doi:10.1109/ASE63991.2025.00310

[21] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and L. Xiang. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. arXiv:2504.02605 doi:10.48550/arXiv.2504.02605

[22] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, J. Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Yudong Zhang. 2025. SWE-bench Goes Live! arXiv:2505.23419 doi:10.48550/arXiv.2505.23419